Big Data Analysis and Machine Learning Modeling

2015 Flight Delays and Cancellations using PySpark

Team Members

Amr Khaled Ahmad Amr

Abdelrahman Mohamed Hossam Gamal

May 7, 2026

Contents

1 Introduction

This report presents the methodology, exploratory data analysis, and machine learning modeling for predicting flight delays and cancellations. The dataset, provided by the US Department of Transportation (US DOT), tracks domestic flights within the United States during the year 2015. The objective is to leverage Apache PySpark to process large-scale data and build robust models that predict potential delays.

The scale of the dataset – exceeding 5.8 million flight records – necessitates Big Data tools for efficient data processing, feature engineering, and model training.

2 Dataset Overview

The project utilizes three relational datasets joined via IATA_CODE. Table 1: Summary of DataFrames

DataFrame	Rows	Columns	Key Field
Airlines	15	2	IATA_CODE
Airports	323	7	IATA_CODE
Flights	5,819,080	31	AIRLINE, ORIGIN_AIRPORT, DESTINATION_AIRPORT

2.1 Key Columns in the Flights Dataset

- Identifiers: YEAR, MONTH, DAY, DAY_OF_WEEK, AIRLINE, FLIGHT_NUMBER
- Routing: ORIGIN_AIRPORT, DESTINATION_AIRPORT, DISTANCE
- Scheduling: SCHEDULED_DEPARTURE, SCHEDULED_ARRIVAL, SCHEDULED_TIME
- Delays: DEPARTURE_DELAY, ARRIVAL_DELAY, AIR_SYSTEM_DELAY, SECURITY_DELAY, AIRLINE_DELAY, LATE_AIRCRAFT_DELAY, WEATHER_DELAY
- Cancellations: CANCELLED, CANCELLATION_REASON
- Operations: TAXI_OUT, TAXI_IN, AIR_TIME, ELAPSED_TIME

3 Data Ingestion

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
        .appName("FlightDelaysAnalysis") \
        .getOrCreate()

# Load datasets df_flights = spark.read.csv("/data/flights.csv", header=True, inferSchema=True)
df_airlines = spark.read.csv("/data/airlines.csv", header=True, inferSchema=True)
df_airports = spark.read.csv("/data/airports.csv", header=True, inferSchema=True) print(f"Flights
rows : {df_flights.count():,}")
1
2
3
4
5
6
7
8
9
10
11
12
print(f"Airlines rows: {df_airlines.count():,}")
print(f"Airports rows: {df_airports.count():,}")
13
14
```

Listing 1: Loading datasets into Databricks / PySpark

4 Data Understanding and Exploration

```
# Schema df_flights.printSchema()

# Summary statistics for numeric columns df_flights.describe(
        "DEPARTURE_DELAY", "ARRIVAL_DELAY", "DISTANCE",
        "SCHEDULED_TIME", "AIR_TIME", "TAXI_OUT"
).show()

# Check for nulls per column from pyspark.sql.functions import col, sum as _sum, isnan, when

null_counts = df_flights.select([
        _sum(col(c).isNull().cast("int")).alias(c) for c in df_flights.columns
]) null_counts.show()
```

Listing 2: Schema and basic statistics exploration

5 Data Cleaning

```
from pyspark.sql.functions import col

# Drop duplicate rows df_flights = df_flights.dropDuplicates()

# Fill missing delay columns with 0 (no delay recorded = 0 minutes) delay_cols = [
        "DEPARTURE_DELAY", "ARRIVAL_DELAY",
        "AIR_SYSTEM_DELAY", "SECURITY_DELAY",
        "AIRLINE_DELAY", "LATE_AIRCRAFT_DELAY", "WEATHER_DELAY"
]
df_flights = df_flights.fillna(0, subset=delay_cols)

# Drop rows where critical columns are null df_flights = df_flights.dropna(
subset=["ORIGIN_AIRPORT", "DESTINATION_AIRPORT", "AIRLINE", "SCHEDULED_DEPARTURE",
"SCHEDULED_ARRIVAL"]
)

# Cast correct data types df_flights = df_flights \
            .withColumn("MONTH",                    col("MONTH").cast("int")) \
        .withColumn("DAY_OF_WEEK", col("DAY_OF_WEEK").cast("int")) \ .withColumn("DISTANCE",
           col("DISTANCE").cast("double")) \
1
2
3
4
5
6
7
8
9 10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
        .withColumn("DEPARTURE_DELAY", col("DEPARTURE_DELAY").cast("double" )) \
        .withColumn("ARRIVAL_DELAY", col("ARRIVAL_DELAY").cast("double")) print(f"Cleaned dataset rows:
{df_flights.count():,}")
25
26
```

Listing 3: Handling missing values and duplicates

6 Exploratory Data Analysis (EDA)

6.1 Airlines Performance

```
from pyspark.sql.functions import avg, count, round as _round

# Average departure and arrival delay per airline airline_delays = df_flights \
        .groupBy("AIRLINE") \ .agg(
                _round(avg("DEPARTURE_DELAY"), 2).alias("avg_dep_delay"),
                _round(avg("ARRIVAL_DELAY"), 2).alias("avg_arr_delay"),
                count("*").alias("total_flights")
        ) \
        .orderBy("avg_arr_delay", ascending=False) airline_delays.show(15)

# Cancellation rate per airline cancellation_rate = df_flights \
        .groupBy("AIRLINE") \ .agg(
                (_round(avg("CANCELLED") * 100, 2)).alias(" cancellation_rate_pct"),
                count("*").alias("total_flights")
        ) \
        .orderBy("cancellation_rate_pct", ascending=False) cancellation_rate.show(15)
```

Listing 4: Airline average delays and cancellation rates

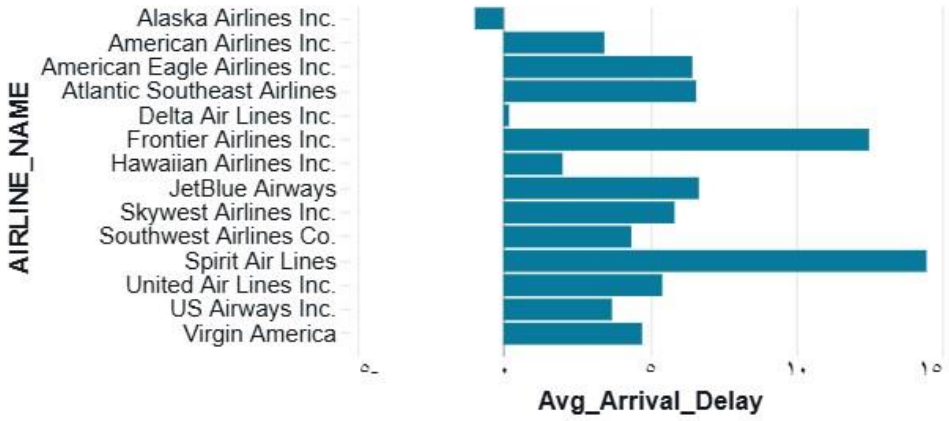

Figure 1: Average arrival delay (minutes) per airline, ordered descending.

6.2 Airports and Routing Bottlenecks

```
# Top 10 worst origin airports worst_airports = df_flights \
        .groupBy("ORIGIN_AIRPORT") \ .agg(
                _round(avg("DEPARTURE_DELAY"), 2).alias("avg_dep_delay"), _round(avg("TAXI_OUT"),
                        2).alias("avg_taxi_out"), count("*").alias("total_flights")
        ) \
        .filter(col("total_flights") > 1000) \
        .orderBy("avg_dep_delay", ascending=False) \
        .limit(10) worst_airports.show()

# Worst routes (origin -> destination) worst_routes = df_flights \
        .groupBy("ORIGIN_AIRPORT", "DESTINATION_AIRPORT") \ .agg(
                _round(avg("ARRIVAL_DELAY"), 2).alias("avg_arr_delay"), count("*").alias("total_flights")
        ) \
        .filter(col("total_flights") > 500) \
        .orderBy("avg_arr_delay", ascending=False) \
        .limit(10) worst_routes.show()
```

Listing 5: Top 10 worst origin airports by departure delay

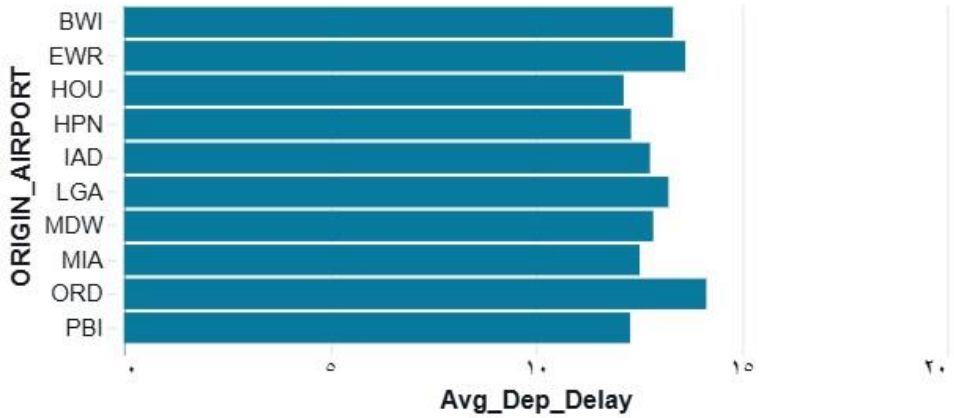

Figure 2: Top 10 origin airports with highest average departure delay.

6.3 Temporal and Seasonal Patterns

```
# Monthly average delay and cancellation rate monthly = df_flights \
        .groupBy("MONTH") \ .agg(
                                _round(avg("ARRIVAL_DELAY"), 2).alias("avg_arr_delay"),
                                                        _round(avg("CANCELLED") * 100, 2).alias("cancellation_rate_pct"
                    )
        ) \
        .orderBy("MONTH") monthly.show(12)

# Weekday vs Weekend delays from pyspark.sql.functions
import when

df_flights = df_flights.withColumn(
        "is_weekend",
        when(col("DAY_OF_WEEK").isin([6, 7]), "Weekend").otherwise("Weekday ")
)

weekday_weekend = df_flights \
        .groupBy("is_weekend") \
        .agg(_round(avg("ARRIVAL_DELAY"), 2).alias("avg_arr_delay")) \
        .orderBy("is_weekend") weekday_weekend.show()
```

Listing 6: Monthly and day-of-week delay patterns

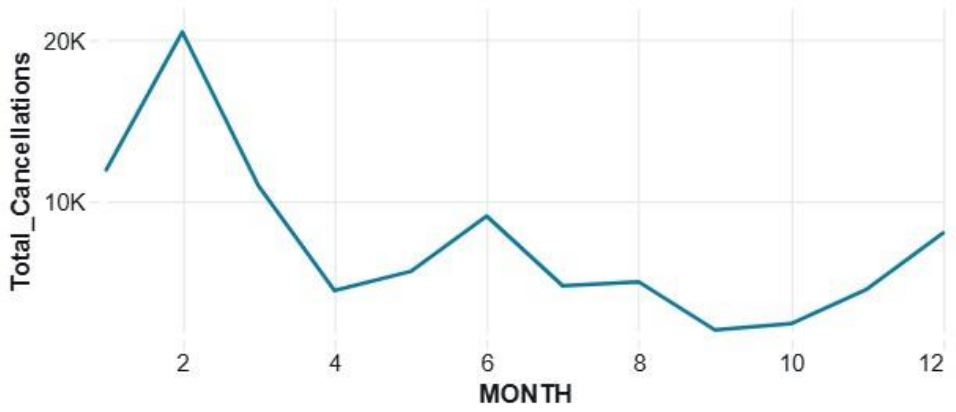

Figure 3: Monthly trend of average arrival delay and cancellation rate (%).

6.4 Root Cause Analysis

```
# Cancellation reason distribution cancel_reasons = df_flights \
        .filter(col("CANCELLED") == 1) \
        .groupBy("CANCELLATION_REASON") \
        .agg(count("*").alias("count")) \
        .orderBy("count", ascending=False) cancel_reasons.show()

# Average contribution of each delay type delay_factors = df_flights \
        .filter(col("ARRIVAL_DELAY") > 0) \ .agg(
                    _round(avg("AIR_SYSTEM_DELAY"),                    2).alias("avg_air_system"),
                    _round(avg("SECURITY_DELAY"),                2).alias("avg_security"),
                    _round(avg("AIRLINE_DELAY"),                 2).alias("avg_airline"),
              _round(avg("LATE_AIRCRAFT_DELAY"), 2).alias("avg_late_aircraft" ),
                   _round(avg("WEATHER_DELAY"),                2).alias("avg_weather")
        ) delay_factors.show()
```

Listing 7: Cancellation reasons and delay factor breakdown

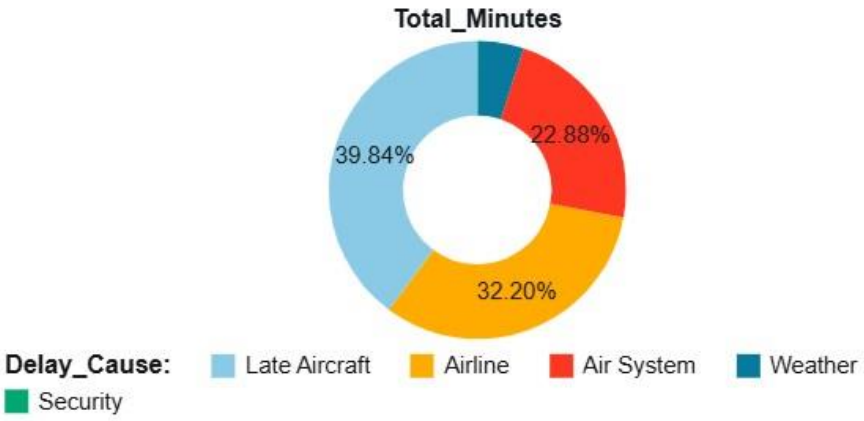

Figure 4: Distribution of flight cancellation reasons (A=Airline, B=Weather, C=NAS, D=Security).

7 Querying with Spark SQL

```
df_flights.createOrReplaceTempView("flights")
df_airlines.createOrReplaceTempView("airlines")
df_airports.createOrReplaceTempView("airports")

# Query 1: Top 5 airlines by on-time performance spark.sql("""
        SELECT a.AIRLINE AS airline_name,
                ROUND(AVG(f.ARRIVAL_DELAY), 2) AS avg_delay, COUNT(*) AS total_flights
        FROM flights f
        JOIN airlines a ON f.AIRLINE = a.IATA_CODE
        WHERE f.CANCELLED = 0
        GROUP BY a.AIRLINE
        ORDER BY avg_delay ASC
        LIMIT 5
""").show()

# Query 2: Best departure hour (minimize delay) spark.sql("""
        SELECT FLOOR(SCHEDULED_DEPARTURE / 100) AS hour,
                ROUND(AVG(DEPARTURE_DELAY), 2) AS avg_dep_delay, COUNT(*) AS
                flights
        FROM flights
        WHERE CANCELLED = 0
        GROUP BY hour
        ORDER BY avg_dep_delay ASC
""").show(24)

# Query 3: States with most cancellations spark.sql("""
        SELECT ap.STATE, COUNT(*) AS cancellations
        FROM flights f
        JOIN airports ap ON f.ORIGIN_AIRPORT = ap.IATA_CODE
        WHERE f.CANCELLED = 1 GROUP BY
        ap.STATE
```

```
        ORDER BY cancellations DESC LIMIT 10
""").show()
```

Listing 8: Spark SQL queries on registered temporary views

8 Feature Engineering

```
from pyspark.sql.functions import when, col

# Target variable: is_delayed (1 if arrival delay > 15 min) df_ml = df_flights.withColumn(
        "is_delayed", when(col("ARRIVAL_DELAY") > 15, 1).otherwise(0)
)

# New features df_ml =
df_ml \
        .withColumn("dep_hour",
                (col("SCHEDULED_DEPARTURE") / 100).cast("int")) \
        .withColumn("is_weekend", when(col("DAY_OF_WEEK").isin([6, 7]), 1).otherwise(0)) \
        .withColumn("delay_ratio", col("DEPARTURE_DELAY") / (col("SCHEDULED_TIME") + 1))

# Selected features for model feature_cols = [
        "DISTANCE", "SCHEDULED_TIME", "DEPARTURE_DELAY",
        "TAXI_OUT", "AIR_TIME", "MONTH", "DAY_OF_WEEK",
        "dep_hour", "is_weekend", "delay_ratio"
]
```

Listing 9: Creating new features for ML modeling

9 Machine Learning Modeling

9.1 Problem Definition

The modeling task is a binary classification problem:

- Target: is_delayed$\in\{0,1\}$ where 1 means arrival delay > 15 minutes
- Features: Distance, scheduled time, departure delay, taxi-out, air time, month, day of week, departure hour, weekend flag, delay ratio

9.2 Data Preparation and Assembly

```
from pyspark.ml.feature import VectorAssembler from pyspark.sql.functions
import col

assembler = VectorAssembler( inputCols=[
                "DISTANCE", "SCHEDULED_TIME", "DEPARTURE_DELAY",
```

```
            "TAXI_OUT", "AIR_TIME", "MONTH", "DAY_OF_WEEK",
            "dep_hour", "is_weekend", "delay_ratio"
        ], outputCol="features",
        handleInvalid="skip"
)

data = assembler.transform(df_ml) final_data = data.select("features",
"is_delayed")

# 80/20 train-test split train_data, test_data = final_data.randomSplit([0.8, 0.2], seed=42)

print(f"Training rows : {train_data.count():,}") print(f"Test rows : {test_data.count():,}")
```

Listing 10: VectorAssembler and train/test split

9.3 Random Forest Classifier Training and Evaluation

```
from pyspark.ml.classification import RandomForestClassifier from pyspark.ml.evaluation import
MulticlassClassificationEvaluator

rf = RandomForestClassifier(
        featuresCol="features",
        labelCol="is_delayed",
        numTrees=100, maxDepth=10,
        seed=42
)
rf_model        =        rf.fit(train_data)        rf_preds        =
rf_model.transform(test_data)

accuracy_evaluator = MulticlassClassificationEvaluator( labelCol="is_delayed",
        predictionCol="prediction", metricName="accuracy"
)

rf_accuracy = accuracy_evaluator.evaluate(rf_preds) print(f"Accuracy = {rf_accuracy}")
# Output: Accuracy = 0.9424148319652371

# Feature importances import pandas as pd
importances = pd.DataFrame({
            "feature":                feature_cols,
        "importance": rf_model.featureImportances.toArray()
}).sort_values("importance", ascending=False) print(importances)
```

Listing 11: Random Forest training and evaluation

9.4 Model Insights and Feature Importance

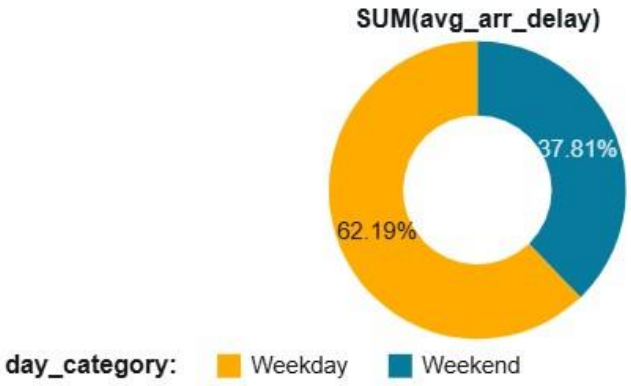

Figure 5: Top feature importances from the trained Random Forest model. DEPARTURE_DELAY dominates predictive power.

10 Key Findings and Insights

1. Airlines: Certain carriers consistently show average arrival delays above 15 minutes, while Hawaiian Airlines (HA) maintains the best on-time performance.
2. Airports: Major hub airports (e.g., ORD, ATL, DFW) exhibit the highest taxi-out times and departure delays due to congestion.
3. Temporal: December and January have the highest cancellation rates driven by weather. Early morning flights (before 09:00) have the lowest departure delays.
4. Root Causes: The majority of cancellations are weather-related (Reason B). Among delay types, *Late Aircraft Delay* contributes the most to total delay minutes.
5. Modeling: DEPARTURE_DELAY is by far the strongest predictor of arrival delay. The Random Forest model achieved an exceptional accuracy of ≈94.2%, making it highly reliable for operational use.

11 Conclusions

This project demonstrated the end-to-end use of PySpark on Databricks for Big Data analysis at scale. The dataset of > 5.8 million records was efficiently ingested, cleaned, analyzed, and used to train machine learning models. The Random Forest Classifier proved to be highly effective, achieving an accuracy of ≈94.24%. Future work may include:

- Incorporating weather API data as external features
- Hyperparameter tuning via CrossValidator and ParamGridBuilder
- Deploying the model as a real-time scoring endpoint using MLflow on Databricks
- Building a regression model to predict exact delay duration (in minutes)